

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Максимова Ольга

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. реализовать Dockerfile для каждого сервиса;
2. написать общий docker-compose.yml;
3. настроить сетевое взаимодействие между сервисами.

Ход работы

Для каждого микросервиса был создан Dockerfile. Пример для user:

```
1 FROM node:20-alpine AS builder
2 WORKDIR /app
3
4 COPY common/package*.json ./common/
5 COPY user/package*.json ./user/
6
7 COPY common ./common
8 COPY user ./user
9
10 RUN npm install --prefix ./common
11 RUN npm install --prefix ./user
12
13 WORKDIR /app/common
14 RUN npm run build
15
16 WORKDIR /app/user
17 RUN npm run build
18
19
20 FROM node:20-alpine AS prod
21 ENV NODE_ENV=production
22 WORKDIR /app
23
24 COPY common/package*.json ./common/
25 COPY user/package*.json ./user/
26
27 RUN cd common && npm install --omit=dev --omit=optional \
28     && cd ../user && npm install --omit=dev --omit=optional
29
30 COPY --from=builder /app/common/dist ./common/dist
31 COPY --from=builder /app/user/dist ./user/dist
32
33 WORKDIR /app/user
34 CMD ["node", "dist/index.js"]
```

В стадии builder берется образ node:20-alpine, задается рабочая директория /app, затем копируются package*.json и исходники для common и user. После этого для каждого пакета отдельно ставятся зависимости, а потом запускается сборка через npm run build в папках common и user.

Во второй стадии снова используется node:20-alpine, выставляется NODE_ENV=production, создается /app. Затем копируются только package*.json

для common и user, после чего для обоих пакетов отдельно ставятся production-зависимости через `npm install --omit=dev --omit=optional`.

Дальше из builder копируются только папки dist для common и user, и контейнер запускает `node dist/index.js` из user. То есть в финальном образе нет исходников и dev-зависимостей, только то, что нужно для запуска.

Также был создан общий `docker-compose.yml`.

```
Run All Services
services:
  Run Service
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.1
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    networks:
      - rent-net

  Run Service
  kafka:
    image: confluentinc/cp-kafka:7.5.1
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
    depends_on:
      - zookeeper
    healthcheck:
      test: ["CMD-SHELL", "kafka-broker-api-versions --bootstrap-server localhost:9092 | grep -q 'ApiVersion' || exit 1"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 20s
    networks:
      - rent-net
```

zookeeper запускает контейнер `confluentinc/cp-zookeeper:7.5.1`, пробрасывает порт 2181 и задает два параметра: клиентский порт и tick time.

kafka запускает брокер `confluentinc/cp-kafka:7.5.1`, который подключается к Zookeeper через `zookeeper:2181`. `KAFKA_LISTENERS` говорит, на каком адресе и порту брокер реально слушает внутри контейнера, а

KAFKA_ADVERTISED_LISTENERS – какой адрес Kafka будет отдавать клиентам в metadata.

KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false" отключает автоматическое создание тем, чтобы темы создавались только явно.

healthcheck проверяет, что брокер уже отвечает, а depends_on указывает, что Kafka стартует после Zookeeper. depends_on управляет порядком запуска, а healthcheck позволяет дождаться реальной готовности сервиса, а не просто факта запуска контейнера.

```

> Run Service
kafka-init:
  image: confluentinc/cp-kafka:7.5.1
  depends_on:
    kafka:
      condition: service_healthy
  command: >
    sh -c '
      kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic rent.events --partitions 1 --replication-factor 1 &&
      kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic payment.events --partitions 1 --replication-factor 1 &&
      kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic accommodation.events --partitions 1 --replication-factor 1 &&
      kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic user.events --partitions 1 --replication-factor 1 &&
      kafka-topics --bootstrap-server kafka:9092 --create --if-not-exists --topic message.events --partitions 1 --replication-factor 1
    '
  restart: "no"
  networks:
    - rent-net

```

kafka-init – это одноразовый контейнер, который ждет, пока Kafka станет healthy, и затем создает топики rent.events, payment.events, accommodation.events, user.events, message.events. Команда kafka-topics --create --if-not-exists делает так, что если тема уже есть, контейнер не упадет на повторном запуске. После выполнения этих команд контейнер завершится, потому что restart: "no" запрещает ему перезапускаться.

```
>Run Service
user:
  build:
    context: .
    dockerfile: user/Dockerfile
  env_file: ./user/.env
  ports:
    - "8001:8001"
  depends_on:
    kafka-init:
      condition: service_completed_successfully
  networks:
    - rent-net
```

```
>Run Service
accommodation:
  build:
    context: .
    dockerfile: accommodation/Dockerfile
  env_file: ./accommodation/.env
  ports:
    - "8000:8000"
  depends_on:
    kafka-init:
      condition: service_completed_successfully
  networks:
    - rent-net
```

```
>Run Service
payment:
  build:
    context: .
    dockerfile: payment/Dockerfile
  env_file: ./payment/.env
  ports:
    - "8003:8003"
  depends_on:
    kafka-init:
      condition: service_completed_successfully
  networks:
    - rent-net
```

```
>Run Service
rent:
  build:
    context: .
    dockerfile: rent/Dockerfile
  env_file: ./rent/.env
  ports:
    - "8002:8002"
  depends_on:
    kafka-init:
      condition: service_completed_successfully
  networks:
    - rent-net
```

```
>Run Service
message:
  build:
    context: .
    dockerfile: message/Dockerfile
  env_file: ./message/.env
  ports:
    - "8004:8004"
  depends_on:
    kafka-init:
      condition: service_completed_successfully
  networks:
    - rent-net
```

Дальше идут блоки для запуска микросервисов: `user`, `accommodation`, `payment`, `rent`, `message`. Каждый сервис собирается из своего `Dockerfile`, получает переменные окружения из собственного `.env`, публикует порт на хост и запускается только после успешного завершения `kafka-init`.

`build.context: .` означает, что Docker берёт весь текущий каталог как контекст сборки. `dockerfile: user/Dockerfile` говорит, что для сервиса `user` нужно использовать именно этот `Dockerfile` внутри контекста.

`env_file: ./user/.env` подставляет переменные из этого файла внутрь контейнера сервиса `user`.

`depends_on: kafka-init: condition: service_completed_successfully` означает, что сервис не будет стартовать, пока контейнер `kafka-init` не завершится успешно с кодом 0.

```

▷ Run Service
message-db:
  image: postgres:17
  restart: always
  environment:
    POSTGRES_USER: messagedb
    POSTGRES_PASSWORD: messagedb
    POSTGRES_DB: messagedb
  ports:
    - "15435:5432"
  volumes:
    - message-db-data:/var/lib/postgresql/data
  networks:
    - rent-net

▷ Run Service
rent-db:
  image: postgres:17
  restart: always
  environment:
    POSTGRES_USER: rentdb
    POSTGRES_PASSWORD: rentdb
    POSTGRES_DB: rentdb
  ports:
    - "15436:5432"
  volumes:
    - rent-db-data:/var/lib/postgresql/data
  networks:
    - rent-net

▷ Run Service
user-db:
  image: postgres:17
  restart: always
  environment:
    POSTGRES_USER: userdb
    POSTGRES_PASSWORD: userdb
    POSTGRES_DB: userdb
  ports:
    - "15434:5432"
  volumes:
    - user-db-data:/var/lib/postgresql/data
  networks:
    - rent-net

▷ Run Service
payment-db:
  image: postgres:17
  restart: always
  environment:
    POSTGRES_USER: paymentdb
    POSTGRES_PASSWORD: paymentdb
    POSTGRES_DB: paymentdb
  ports:
    - "15433:5432"
  volumes:
    - payment-db-data:/var/lib/postgresql/data
  networks:
    - rent-net

▷ Run Service
accommodation-db:
  image: postgres:17
  restart: always
  environment:
    POSTGRES_USER: accommodationdb
    POSTGRES_PASSWORD: accommodationdb
    POSTGRES_DB: accommodationdb
  ports:
    - "15437:5432"
  volumes:
    - accommodation-db-data:/var/lib/postgresql/data
  networks:
    - rent-net

volumes:
  payment-db-data:
  rent-db-data:
  user-db-data:
  accommodation-db-data:
  message-db-data:

networks:
  rent-net:
    name: rent-network

```

Дальше создаются отдельные PostgreSQL-контейнеры – по одному на каждый микросервис. Каждый контейнер хранит свои данные в отдельном volume, а наружу проброшен свой порт, чтобы к базе можно было подключаться с хоста.

message-db, rent-db, user-db, payment-db, accommodation-db – это одинаково устроенные контейнеры postgres:17. В каждом задаются пользователь, пароль и имя базы через POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, затем база сохраняется в volume по пути /var/lib/postgresql/data.

Внизу volumes: объявлены имена томов payment-db-data, rent-db-data и так далее.

В итоге:

```
✓ accommodation Built
✓ message Built
✓ payment Built
✓ rent Built
✓ user Built
✓ Network rent-network Created
✓ Container lab2-rent-db-1 Recreated
✓ Container lab2-payment-db-1 Recreated
✓ Container lab2-user-db-1 Recreated
✓ Container lab2-accommodation-db-1 Recreated
✓ Container lab2-message-db-1 Recreated
✓ Container kafka Recreated
✓ Container zookeeper Recreated
✓ Container lab2-kafka-init-1 Recreated
✓ Container lab2-user-1 Recreated
✓ Container lab2-payment-1 Recreated
✓ Container lab2-rent-1 Recreated
✓ Container lab2-message-1 Recreated
✓ Container lab2-accommodation-1 Recreated
```

Вывод

Были созданы Dockerfile для каждого микросервисов, общий docker-compose.yml и обеспечено сетевое взаимодействие между сервисами.